

Technical Assessment

Shopify + WordPress/WooCommerce + Custom CRM

Candidate Technical Development Test

Recommended development time: 4–6 hours

Reference material: Provided PDF design/specification

1. Assessment Overview

You are required to develop a small, connected e-commerce ecosystem based on the design and functional requirements provided in the attached PDF. The assessment consists of a custom WordPress/WooCommerce store, a custom Shopify store, a 1,000-product catalogue, and a custom CRM that receives orders from both platforms.

- Custom WordPress/WooCommerce storefront
- Custom Shopify storefront/theme
- Shop page containing 1,000 imported sample products
- Custom CRM for customers and orders
- Shopify → CRM order integration
- WooCommerce → CRM order integration
- API, webhook, validation, security, error handling, and duplicate protection

2. Reference Design

The attached PDF is the primary reference for the visual design and page structure. Translate the design into functional, responsive interfaces. Where the PDF does not specify an implementation detail, make a reasonable technical decision and document it.

- Layout and component structure
- Typography
- Navigation
- Product presentation
- Responsive behavior
- General UI/UX

3. Part A — Custom WordPress/WooCommerce Store

Develop a custom WordPress/WooCommerce implementation based on the supplied design.

Required Pages

- Homepage
- Shop
- Product detail
- Product category
- Cart
- Checkout
- Customer account
- Search results

Technical Requirements

- WordPress + WooCommerce
- PHP, HTML, CSS, and JavaScript

- Custom theme for the core storefront
- Appropriate WordPress template hierarchy
- Actions and filters
- WooCommerce hooks and template overrides
- WordPress/WooCommerce APIs where appropriate
- Secure input handling and proper asset loading
- Reusable components and maintainable code

Do not build the entire website using Elementor, Divi, WPBakery, or another visual page builder. Third-party plugins may be used where justified; document significant plugins and their purpose.

4. Part B — Custom Shopify Store

Develop a custom Shopify storefront based on the same supplied PDF.

Required Pages

- Homepage
- Shop/Collection
- Product page
- Search
- Cart

Technical Requirements

- Shopify Online Store 2.0 architecture
- Liquid
- Sections and blocks
- Snippets
- JSON templates
- Theme settings
- Metafields where appropriate
- JavaScript
- Shopify APIs where required

Major homepage sections should be configurable through the Shopify Theme Editor rather than hard-coded.

5. Part C — 1,000 Product Dataset

Create or use the supplied dataset representing 1,000 realistic products and import the same logical catalogue into both Shopify and WooCommerce.

Product Data

- Product ID
- SKU
- Product name
- Description
- Price
- Sale price
- Category and subcategory
- Brand/vendor
- Tags
- Stock quantity

- Product images
- Variants
- Attributes
- Specifications
- SEO title
- SEO description

Import Requirements

- Products and variants must be correctly associated
- SKUs, categories, tags, images, pricing, inventory and descriptions must be mapped
- Custom product information must be preserved where applicable
- The 1,000 products must not be manually created
- Document the import/transformation process

6. Shop Page

The Shop page must display the imported catalogue and remain practical with 1,000 products.

- Product listing/grid
- Pagination or efficient incremental loading
- Search
- Category filtering
- Attribute/product filtering
- Price filtering
- Sorting
- Product availability
- Responsive layout

Do not unnecessarily load all 1,000 products into the browser at once. Use appropriate server-side querying, API pagination, or equivalent mechanisms.

7. Part D — Custom CRM

Develop a separate CRM application that acts as the central order-management system for both stores.

CRM Dashboard

- Total orders
- Total revenue
- Pending orders
- Completed orders
- Cancelled orders
- Recent orders

CRM Orders

- Internal order ID
- External order ID
- Order source
- Customer
- Email
- Phone

- Billing address
- Shipping address
- Products
- Quantities
- Subtotal
- Discount
- Tax
- Shipping
- Total
- Payment status
- Order status
- Created date

Clearly identify the source of every order as Shopify or WooCommerce.

8. CRM — Customers

- View customers
- Search customers
- View customer details
- View customer order history
- Avoid unnecessary duplicate customer records

9. Part E — Shopify → CRM Integration

When an order is created on Shopify, it must automatically reach the CRM using an appropriate webhook/API integration.

Shopify Order → Webhook/API → CRM API → Validation → Database → CRM Order

Store the Shopify external order identifier in the CRM.

10. Part F — WooCommerce → CRM Integration

When an order is created in WooCommerce, it must automatically reach the CRM.

WooCommerce Order → Webhook/API → CRM API → Validation → Database → CRM Order

The CRM must identify the order as originating from WooCommerce.

11. API Requirements

The CRM must expose an appropriate REST API. The exact route structure may differ if the architecture is justified.

```
POST /api/orders
GET /api/orders
GET /api/orders/{id}
PATCH /api/orders/{id}

GET /api/customers
GET /api/customers/{id}

GET /api/products
```

- Authentication
- Authorization
- Request validation
- Error handling
- Appropriate HTTP status codes
- Pagination
- Filtering

- Logging

12. Webhook Security

External systems must not be able to submit arbitrary order data to the CRM without validation.

- Authentication
- Signature verification where supported
- Request validation
- Input sanitization
- Replay/duplicate protection
- Rate limiting where appropriate

Document your security approach.

13. Duplicate Order Protection

The CRM must handle duplicate webhook/API requests without creating duplicate orders.

Example: Shopify + External Order ID = unique order identity

Apply the same principle to WooCommerce. The integration should be idempotent.

14. Failure Handling

Consider a situation where a store sends an order but the CRM API is temporarily unavailable.

- Detect and log failed synchronization
- Store enough information to retry
- Provide visibility into failed synchronization
- Prevent duplicates when a request is retried

A complex distributed queue is not mandatory unless your architecture requires it, but the failure mode must be understood and addressed.

15. Order Status Synchronization

- Pending
- Processing
- Completed
- Cancelled
- Refunded

Two-way status synchronization between CRM and the stores is optional and will be considered an additional strength.

16. Database Design

Design an appropriate relational database structure for the CRM.

```
Customers
↓
Orders
↓
Order Items
↓
Products

Store
↓
External Order ID
↓
CRM Order
```

Submit the database schema/ERD with the project.

17. Authentication & Access Control

- Login
- Logout
- Protected CRM routes

Basic role-based access control such as Admin and Staff is encouraged.

18. Performance Requirements

The system should be designed with the expectation that the catalogue may grow beyond 1,000 products.

- Database indexing
- Efficient queries
- Pagination
- API pagination
- Image optimization
- Lazy loading
- Caching where appropriate
- Avoid unnecessary API requests
- Avoid unnecessary frontend rendering

Be prepared to explain how the architecture would scale to 10,000+ products and higher order volumes.

19. Responsive Design

- Desktop
- Tablet
- Mobile

The CRM should also provide a usable responsive interface.

20. Code Quality

- Code organization
- Reusability
- Naming conventions
- Separation of concerns
- Maintainability
- Error handling
- Security
- Performance
- Documentation

Avoid unnecessary duplication and unexplained complexity.

21. Deployment

- Working WordPress/WooCommerce environment
- Working Shopify development/test store
- Working CRM environment

The reviewer must be able to test the complete end-to-end workflow.

22. Required Demonstration

1. Place an order on Shopify and verify that it appears automatically in the CRM.
2. Place an order on WooCommerce and verify that it appears automatically in the CRM.
3. Open the CRM customer profile and verify the associated order history.
4. Process/send the same webhook twice and verify that no duplicate order is created.
5. Search and filter the 1,000-product catalogue and demonstrate that the shop page remains functional and performant.

23. Deliverables

Source Code

- WordPress custom theme
- Shopify custom theme
- CRM frontend
- CRM backend
- Product import scripts/tools
- Integration code

Documentation

- Architecture
- Technology stack
- Installation instructions
- Environment variables
- Database structure
- API documentation
- Shopify integration
- WooCommerce integration
- Webhook implementation
- Product import process
- Security considerations
- Performance considerations
- Known limitations

API Documentation

Provide Swagger/OpenAPI documentation or a Postman collection.

Database

Provide database schema, ERD, and migration/seed files where applicable.

Deployment

Provide access to the WordPress store, Shopify store, and CRM.

24. Restrictions

The objective is to evaluate actual development and engineering capability.

Do not

- Use Elementor/Divi/WPBakery to build the complete WordPress website
- Use a pre-built Shopify theme as the primary implementation
- Manually create the 1,000 products
- Hard-code product data into the frontend
- Hard-code orders into the CRM

- Submit screenshots instead of functional implementations
- Copy an existing implementation without understanding it

You may

- Use documented libraries
- Use open-source packages
- Use standard WooCommerce functionality
- Use Shopify APIs
- Use third-party packages where appropriate

Document significant dependencies and explain their purpose.

25. Technical Discussion

After submission, you will be asked to explain your implementation and demonstrate the system.

- CRM architecture
- Database design
- Shopify architecture
- WordPress architecture
- Product import strategy
- API design
- Webhook implementation
- Authentication
- Idempotency
- Error handling
- Scaling to 10,000+ products
- Performance optimization
- Security
- Architectural trade-offs

You should be able to explain the code and technical decisions in your submission.

26. Evaluation Criteria

Area	Weight
Shopify development	20%
WordPress/WooCommerce development	20%
Product import & Shop functionality	15%
CRM development	15%
API & store integrations	15%
Security, performance & error handling	10%
Code quality & documentation	5%
Total	100%

The primary evaluation criterion is working functionality. A visually attractive implementation with poor architecture will not receive a high score. Likewise, technically complex architecture that does not produce a reliable working product will not receive a high score.

27. Expected System

[illegible]

28. Time Limit

Recommended development time: 4–6 hours.

You may make reasonable assumptions where the specification is not explicit. Document those assumptions in your README.